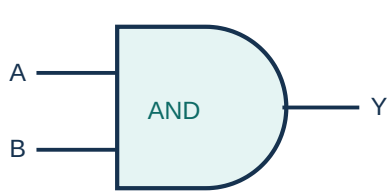
LECTURE NOTES · ORIGINAL TEACHING DRAFT

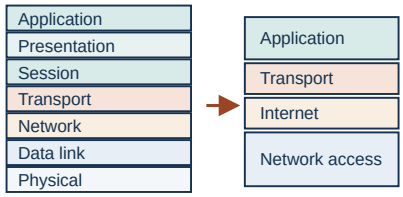
Computer Science

Classes XI and XII

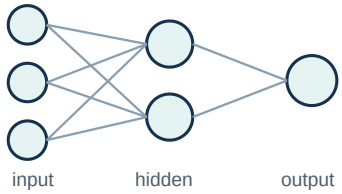
Twelve lectures on how computers represent information, how software is built, how algorithms and Python programs solve problems, how data is stored and analysed, how people and machines interact, and how a small digital idea becomes a first product.



Digital logic · Class XI



OSI and TCP/IP · Class XI



Neural networks · Class XII

These notes follow the learning outcomes of the Class XI and Class XII computer-science units. The explanations, worked examples, and diagrams are written for the lecture; they are not a copy of the textbook. Diagrams are inline SVG. Formulas are set with MathJax. The page is laid out with CSS flexbox, then printed to PDF so the vectors remain sharp.

How to use a lecture

1. Outcome

Read the checklist. If you can already do an item, skim that section and spend time on the one you cannot explain aloud.

2. Diagram

Redraw the figure from memory. A gate, a stack, or an ER diagram that you can redraw is a diagram you understand.

3. Example

Close the notes and repeat the worked example with a new set of numbers. Then answer the checkpoint before you look at the sketch answers.

1. **Computer systems** · XI · logic, SDLC, OSI and TCP/IP
2. **Computational thinking** · XI · strategies, sort, and search
3. **Programming fundamentals** · XI · Python, decisions, loops
4. **Data and analysis** · XI · databases, ER models, Access
5. **Applications and impacts** · XI · AI, IoT, reliable sources
6. **Digital literacy** · XI · inquiry, data collection, ethics
7. **Human–computer interaction** · XII · accessibility, UI/UX, Figma
8. **Algorithms and structures** · XII · traces, Big O, lists, trees
9. **Python in depth** · XII · collections, functions, files
10. **Data analysis with Python** · XII · SQLite, charts, statistics
11. **Impacts of computing** · XII · networks, security, equity
12. **Digital entrepreneurship** · XII · prototype, MVP, beachhead

CLASS XI · LECTURE 1

Computer systems

From two electrical states, to a software process, to a message crossing a network.

1. Distinguish discrete and continuous quantities, and analog and digital representation.
2. Apply Boolean operations, build truth tables, and read logic-gate diagrams.
3. Describe the six phases of the software development life cycle, and choose between Waterfall and Agile.
4. Explain the OSI and TCP/IP models, the job of each layer, and encapsulation.

Discrete, continuous, analog, digital

A **discrete** quantity jumps from one countable value to the next: students in a room, books on a shelf. A **continuous** quantity can fall anywhere inside a range: temperature, height, the position of a car along a road.

An **analog** signal follows a continuous quantity. A microphone voltage rises and falls with the sound. A **digital** signal uses a finite set of symbols. Inside a computer those symbols are two voltages, written 0 and 1. A clock, a calculator, and a processor all reduce their work to patterns of those two states.

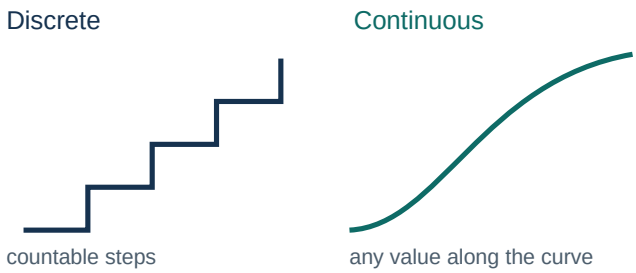


Figure 1. Stairs are discrete. A ramp is continuous.

ONE SENTENCE Digital systems store and process information as symbols from a finite set, and modern computers use the two symbols 0 and 1.

Boolean algebra

Boolean algebra, introduced by George Boole, uses only the values 0 and 1. A letter such as A or B is a variable. An expression combines variables with operations. With n inputs a complete truth table has

$$\text{rows} = 2^n.$$

Two inputs need 4 rows. Three inputs need 8. List the input combinations in binary order so none are missed.

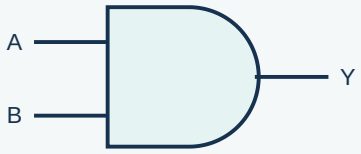
Operation	Reading	Expression	Output is 1 when
AND	both	$Y = A \cdot B$	every input is 1
OR	either	$Y = A + B$	at least one input is 1
NOT	opposite	$Y = \overline{A}$	the single input is 0

A door with two locks is AND: both must be open. A room with two doors is OR: either door is enough. A switch that turns a lamp off when it was on is NOT. NOT has one input; AND and OR have two or more.

Logic gates

A logic gate is a circuit that performs one Boolean operation. Each gate in this course has one output. NAND and NOR are universal: either one, used repeatedly, can build AND, OR, and NOT. XOR is 1 when the inputs differ. XNOR is 1 when they match.

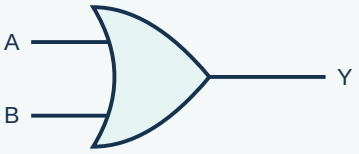
AND · $Y = A \cdot B$



AND gate. The output is 1 only when both inputs are 1.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

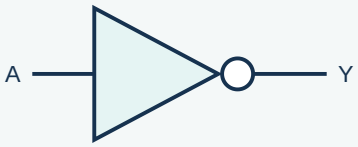
OR · $Y = A + B$



OR gate. The output is 1 when either input is 1.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

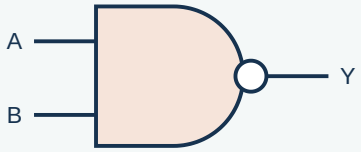
NOT · $Y = \overline{A}$



NOT gate. The bubble inverts the single input.

A	Y
0	1
1	0

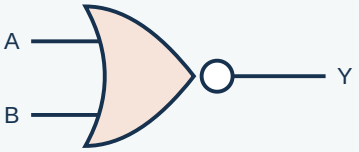
NAND · $Y = \overline{A \cdot B}$



NAND gate. AND followed by a bubble. The output is 0 only when both inputs are 1.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

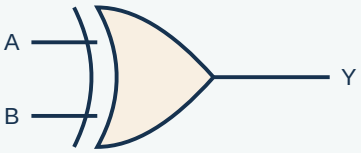
NOR · $Y = \overline{A + B}$



NOR gate. OR followed by a bubble. The output is 1 only when both inputs are 0.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XOR · $Y = A \oplus B$



XOR gate. The extra curve marks exclusive OR. The output is 1 when the inputs differ.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR is XOR followed by a bubble: $Y = \overline{A \oplus B}$. It is 1 only on the rows 00 and 11.

The small circle, or bubble, means inversion. NAND is AND then NOT. NOR is OR then NOT. Read a bubble on the output before you memorise the table: build the inner gate, then flip every output bit.

Worked circuit: a night alarm

A shop alarm should sound when it is night and either the door or the window opens. Let D be the door sensor, W the window sensor, and N the night switch.

$$Y = (D + W) \cdot N$$

Build it as an OR gate feeding an AND gate. The truth table has $2^3 = 8$ rows. Y is 1 only for $(D, W, N) = (1, 0, 1)$, $(0, 1, 1)$, and $(1, 1, 1)$. In daylight $N = 0$, so $Y = 0$ even if a sensor trips. That is the circuit the Boolean expression demanded.

LAB · LOGISIM EVOLUTION

The textbook calls the simulator LogiSim. The steps below build the two-input AND circuit $Y = A \cdot B$.

1. Start a project, then use File → Save As. The file ends in `.circ`.
2. In the explorer, open Gates and place an AND gate on the canvas.
3. Place two input pins on the left and one output pin on the right. Label them A, B, and Y.
4. Draw wires from A and B into the gate and from the gate to Y. Escape returns you to the select tool.
5. Use the poke tool. Click the inputs through 00, 01, 10, and 11. The output must match the AND table.

The software development life cycle

The SDLC is a planned route from a need to working software that can be maintained. The course uses six phases. Each phase has a reason and a deliverable.

1. **Requirement analysis.** Ask users what the system must do. Write the Software Requirements Specification (SRS).
2. **System design.** Decide structure (high-level design), components (low-level design), the database, and the screens. Deliver a design document.
3. **Implementation.** Write readable code, comment the hard parts, and review it with another person.
4. **Testing.** Unit tests check one piece. Integration tests check pieces together. System testing checks the whole product. User acceptance testing is done by the people who will use it.
5. **Deployment.** Release the software to its users.
6. **Maintenance.** Fix faults, adapt to new rules, and improve what real use reveals.

> **Black-box testing** looks only at inputs and outputs. The tester need not see the code. A login trial is black-box: correct details enter, and either the account opens or it does not.
>
> **White-box testing** reads the code and checks that branches and conditions behave. Developers usually do this work.

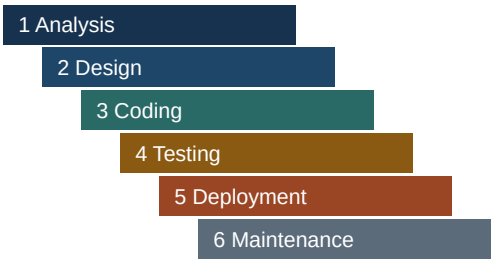


Figure 2. Waterfall finishes one phase before the next begins.

Waterfall and Agile

Waterfall is a strong choice when the requirements are already clear and are unlikely to move. The textbook's library system is that kind of project: the records, the loans, and the reports can be specified before coding starts. The cost of Waterfall is late discovery. Testing sits after coding, so a misunderstood requirement may appear only when the build is large.

Agile, in the Scrum form used for the textbook's school mobile app, expects the requirements to change. Work is divided into short sprints. The textbook case uses two-week sprints. Each sprint has planning, daily stand-up, building, a review of working software, and a retrospective on the team's process. The product backlog is allowed to change after feedback. That fits an app whose students, teachers, and parents discover features as they try them.

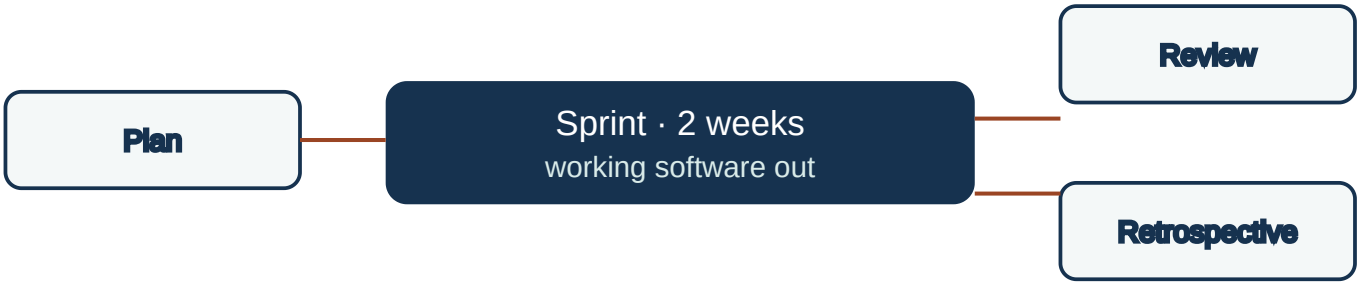


Figure 3. A Scrum sprint returns something usable, then plans the next slice.

	Waterfall	Agile / Scrum
Requirements	Written up front in an SRS	Emerge and change in the backlog
Delivery	One release after the late phases	Working software every sprint
Testing	A dedicated phase after coding	Continuous, inside the sprint
Best fit	Stable, well-understood work	Unclear needs, fast feedback

OSI and TCP/IP

A communication model divides the work of sending a message into layers. Each layer has one job and talks to the layers next to it. The **OSI model** (Open Systems Interconnection), published by ISO, is a seven-layer reference. The **TCP/IP model** is the four-layer model the Internet actually runs. A top-to-bottom memory line for OSI is “All People Seem To Need Data Processing.”

#	OSI layer	Main job	You will meet
7	Application	The service the user touches	HTTP, HTTPS, FTP, SMTP, DNS
6	Presentation	Format, compression, encryption	Translation between data formats
5	Session	Open, keep, and close a dialogue	Session setup and recovery
4	Transport	End-to-end delivery and ports	TCP, UDP
3	Network	Logical addresses and routing	IP, ICMP · IPv4 is 32-bit, IPv6 is 128-bit
2	Data link	Frames, MAC addresses, error check	Ethernet, a switch
1	Physical	Bits as signals on a medium	Cable, fibre, radio, a hub

TCP and UDP

TCP sets up a connection, numbers data so it can be reassembled, and resends what is lost. Web pages, email, and file transfer use it. **UDP** sends datagrams without that handshake. It is lighter, and it is used where a late packet is worse than a missing one: live video, online games, and many DNS queries.

TCP/IP in four layers

Application in TCP/IP covers OSI layers 7, 6, and 5. Transport matches OSI layer 4. Internet matches OSI layer 3. Network Access covers OSI layers 2 and 1. Every device on the Internet uses this stack.

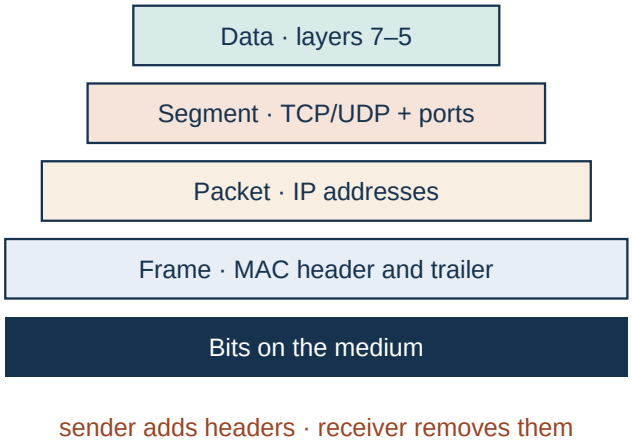


Figure 4. Encapsulation going down. Decapsulation climbs back up.

ONE SENTENCE OSI is the seven-layer map we use to explain networks; TCP/IP is the four-layer stack that carries Internet traffic.

Checkpoint

1. How many rows does a truth table with 4 inputs have?
2. A gate outputs 0 only when both inputs are 1. Name it, and write the expression.
3. Name the SDLC phase whose deliverable is the SRS, and the two testing methods that differ by whether the tester sees the code.
4. Which OSI layer adds port numbers, and which data-unit name belongs to that layer?

Sketch answers: 16 rows. NAND, $Y = \overline{A \cdot B}$. Requirement analysis; black-box and white-box. Transport layer; the segment.

CLASS XI · LECTURE 2

Computational thinking and algorithms

Break the problem down, then choose a search or a sort that fits the data.

1. Use decomposition, pattern recognition, and abstraction on a real problem.
2. State what an algorithm is, how it differs from pseudocode, and what makes one good.
3. Trace bubble sort and selection sort, and linear search and binary search.
4. Choose an algorithm from the shape of the data and the cost of a wrong choice.

Three strategies

Decomposition

Split one hard job into smaller jobs that can be solved and checked on their own. Planning a school trip becomes transport, permissions, food, and a headcount. A bug is easier to find when only one of those jobs is wrong.

Pattern recognition

Look for a repeat. If every pass of a sort compares neighbours, you do not invent a new method for each pair. The same comparison is the pattern; the data values change.

Abstraction

Keep the details that change the result, and set the rest aside. A search cares about the order of the values and the target. It does not care whether those values are roll numbers or prices.

Algorithms and pseudocode

An **algorithm** is a finite sequence of clear steps that produces a result. It is an idea, not a file in a particular language. **Pseudocode** writes that idea in structured English so a programmer can turn it into code. The algorithm says what must happen. The pseudocode shows one readable way to carry it out.

A good algorithm has	Meaning in practice
Input	The data it is allowed to receive is stated.
Output	What it returns, or prints, is stated.
Definiteness	Each step has one meaning. “Sort it somehow” is not a step.
Finiteness	It stops. A loop needs a condition that becomes false.
Effectiveness	Each step is small enough to do, by hand or by a machine.

Later you will judge the same algorithm on three further questions: **correctness** (the output matches the expected result on valid input), **efficiency** (how the work grows as the input grows), and **clarity** (another person can read it and change it).

Bubble sort and selection sort

Both sorts taught here are simple and both do about n^2 comparisons. They are teaching algorithms. On a million items a faster method such as merge sort finishes in far less time. Use these two when the list is short or when the goal is to understand swapping.

Bubble sort

Walk along the list. If two neighbours are out of order, swap them. After one pass the largest item has bubbled to the end. Repeat on the unsorted prefix. If a whole pass makes no swap, the list is already sorted and you may stop.

Selection sort

On pass i, find the smallest item in the unsorted tail and swap it into position i. There is at most one swap per pass. Unlike bubble sort, selection sort does not stop early: it still scans the remaining tail even when the tail is already ordered.

One pass on $[5, 1, 4]$: compare 5 and 1, swap to $[1, 5, 4]$; compare 5 and 4, swap to $[1, 4, 5]$.

On $[5, 1, 4]$: the minimum is 1, swap with 5 to get $[1, 5, 4]$. The minimum of the tail is 4, swap with 5 to get $[1, 4, 5]$.

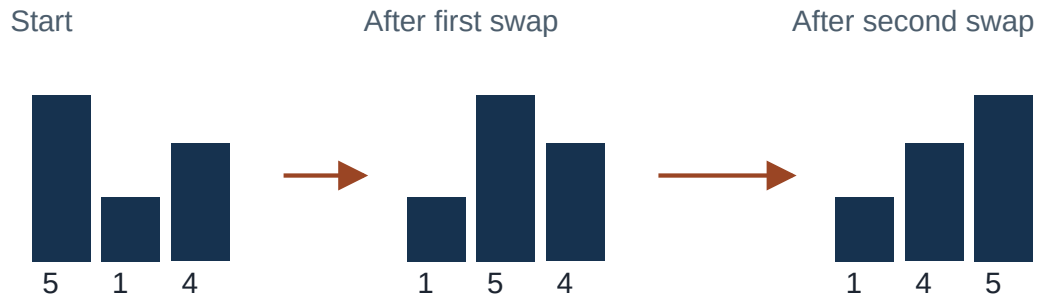


Figure 5. Bubble sort swaps neighbours until the largest bar sits at the end of the pass.

Linear search and binary search

Linear search starts at the first item and checks one by one until it finds the target or runs out of items. The list may be unsorted. On n items the worst case looks at all n of them: order n, written $O(n)$ in the next class. For a dozen items this is often the right tool.

Binary search requires a sorted list. Compare the target with the middle item. If the target is smaller, discard the right half. If it is larger, discard the left half. Repeat. The worst-case number of comparisons grows like the logarithm:

$$\text{steps} \approx \lfloor \log_2 n \rfloor + 1.$$

Worked binary search. Sorted list $[4, 9, 15, 21, 28, 33, 40]$, target 21. Indices run from 0 to 6.

1. $beg = 0$, $end = 6$, $mid = \lfloor (0 + 6)/2 \rfloor = 3$. Value 21. Found.

Target 33 on the same list:

1. $mid = 3$, value 21. 33 is larger, so $beg = 4$.
2. $mid = \lfloor (4 + 6)/2 \rfloor = 5$, value 33. Found in two comparisons, not seven.

An off-by-one error in beg or end is the usual bug. Update the bound to the item *after* or *before* the middle, and stop when the range is empty.

DECISION Unsorted, or a one-off search of a short list: linear search. Sorted, and searched often: binary search. The cost of sorting is paid once; every later search is cheap.

Question	Choose
Is the data unordered, or tiny?	Linear search
Is the data sorted, and will you search it many times?	Binary search
Do you need a sort you can explain with swaps, and the list is short?	Bubble sort or selection sort
Might the list already be nearly sorted, and may you stop early?	Bubble sort, with the no-swap stop
Are swaps expensive, so you want at most one swap per pass?	Selection sort

Checkpoint

1. Name the strategy that throws away details which do not affect the result.
2. After one bubble-sort pass on $[3, 8, 2, 6]$, what is the last value, and why?
3. Why does binary search refuse an unsorted list?

Sketch answers: abstraction. The last value is 8, because each swap moved the current largest item rightward and 8 is the maximum. The half you discard is only safe to discard when you know every value on that side is too small or too large, which sorting guarantees.

CLASS XI · LECTURE 3

Programming fundamentals

Python as a readable language: data, decisions, repetition, libraries, and faults.

1. Place a language on the path from machine code to Python, and name real uses of Python.
2. Use variables, types, input, output, and operators.
3. Direct a program with sequence, selection, and repetition.
4. Call a library and separate a syntax error from a logic error.

Languages and why Python

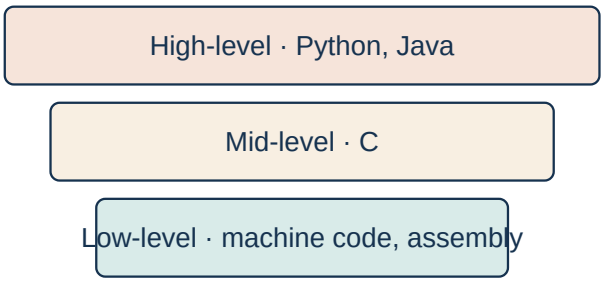


Figure 6. Closer to the machine at the bottom, closer to speech at the top.

A **low-level** language sits near the processor. Machine code is bits. Assembly replaces some of those bits with short mnemonics. A **mid-level** language such as C still lets the programmer talk about memory, but it reads more like a procedure. A **high-level** language hides the machine. Python, Java, and C++ programs describe the problem in words and structures a person can review.

Python shows up in data work, small web tools, automation, and teaching because the same idea is short to write. The editor used in this course is Visual Studio Code: the explorer holds files, the editor holds the program, the terminal runs it, and the debugger stops on a chosen line.

Data, input, and operators

A variable is a name bound to a value. Names describe the value: `score`, not `s2`. Python types you will use immediately are `int`, `float`, `str`, and `bool`.

```
name = input("Student name: ")
marks = int(input("Marks: "))
passed = marks ⩾ 50
print(name, "passed:", passed)
```

`input` always returns text. Arithmetic needs `int` or `float` around it. `print` writes values separated by spaces.

Kind	Symbols	Watch this
Arithmetic	`+ - * / // % **`	`/` is true division. `//` is floor division. `%` is the remainder.
Comparison	`⩵ ≠ > < ⩾ ⩽`	`=` assigns. `⩵` asks a question. The result is `True` or `False`.
Logical	`and or not`	`and` needs both sides true. `or` needs one. `not` flips the value.

Try these before you trust a condition: `4 ⩵ 5 - 1` is true, `55 ⩾ 11 * 5` is true because both sides are 55, and `22 ≠ 23 - 1` is false because both sides are 22.

Sequence, selection, repetition

Sequence is the default: the next line runs after the line above it. **Selection** chooses a block. **Repetition** runs a block while a condition holds, or once for each item.

```
marks = int(input("Marks: "))
if marks ⩾ 85:
    grade = "A"
elif marks ⩾ 70:
    grade = "B"
elif marks ⩾ 50:
    grade = "C"
else:
    grade = "F"
print(grade)
```

The colon opens the block. The indent is the block. `if` alone skips the block when the test fails. `if` / `else` always takes one of the two paths. `elif` tries further tests only when earlier tests failed. Conditions are checked from top to bottom, so put the stricter test first.

```
total = 0
for subject in (78, 64, 91, 55):
    total = total + subject
print("Average:", total / 4)

n = 3
while n > 0:
    print(n)
    n = n - 1
```

A `for` loop is the natural choice when you know the collection. A `while` loop is the natural choice when you know the stopping test. Every `while` needs a line that moves toward that test. Forgetting `n = n - 1` is an infinite loop: the algorithm is no longer finite.

Libraries and debugging

A library is code someone else has tested. A **built-in** library ships with Python. Import it, then use the name you imported.

```
import math
print(math.sqrt(49))
print(math.floor(6.8))
print(math.pi)
```

A **third-party** library is installed separately. Later units use libraries of that kind for tables and charts. Import only what the program uses.

Syntax error

The program does not start. A missing colon, a broken indent, or a bracket that never closes. Read the line number Python prints. Fix that line, then run again.

Runtime error

The program starts and then fails. `int("Ali")` and division by zero are runtime errors. Guard the input, or catch the case before the operation.

Logic error

The program finishes and the answer is wrong. Using `=` inside a comparison, or averaging with the wrong count, is a logic error. Trace the variables by hand.

ONE SENTENCE Sequence does the steps in order, selection chooses a step, and repetition repeats a step until the data or the test says stop.

Checkpoint

1. Why does `int(input(...))` appear so often around numbers?
2. Rewrite “print the first multiple of 7 that is greater than 50” as a `while` idea in one or two lines of reasoning.
3. The program runs and prints 10 for the marks 10 and 6, where the average should be 8. What class of error is that, and what is a likely cause?

Sketch answers: `input` returns a string, and `int` makes a number. Start at 7 and keep adding 7 until the value exceeds 50, then print it. A logic error: the code likely printed one of the marks again, or added the marks and forgot to divide by 2.

CLASS XI · LECTURE 4

Data and analysis

Turn facts into a relational database, then ask it questions.

---

1. Separate data from information, and name the parts of a database.
2. Use keys, cardinality, and referential integrity, including cascade update and delete.
3. Draw an ER model and turn a many-to-many relationship into a schema.
4. Create tables, forms, queries, and reports, and summarise a result with a chart.

From facts to a database

Data are recorded facts: 80, 75, 90, 70, 85. **Information** is data after it has been organised for a decision: the class average is 80. A **database** stores related data so it can be retrieved without copying the same fact into a dozen files. A **DBMS** is the software that creates, protects, and queries that database. Microsoft Access is the DBMS used for the practical work in this unit.

Field

One fact about one thing: a student’s name. A field has a data type.

Record

All the fields for one thing: one student’s row.

Table

The records of one kind of thing. A database is the set of related tables.

Choose types on purpose. Short Text for names, Number for ages and marks, Date/Time for a loan date, Yes/No for a Boolean flag, AutoNumber for a key the DBMS can mint itself. Design view is where you set the type, a required field, a validation rule, and the primary key. Datasheet view looks like a grid and is a fast way to type the first rows once the design is settled.

Keys, relationships, integrity

A **primary key** identifies one record and does not repeat. A **foreign key** is a field in one table that stores the primary key of another table. That is how a row in Loans points at exactly one row in Students.

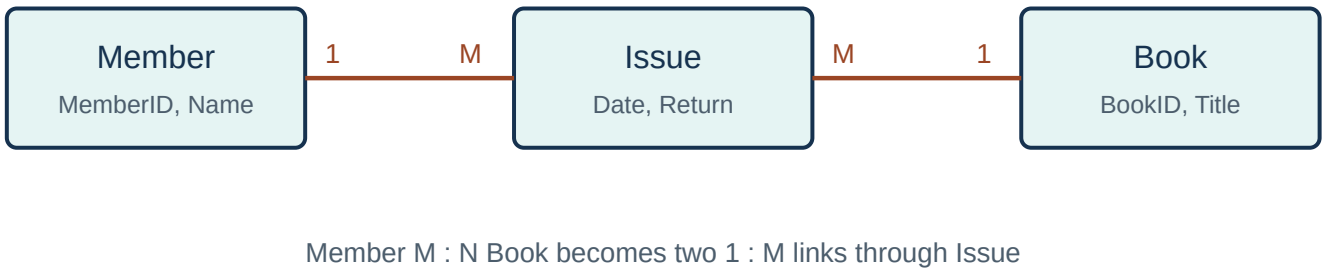
Cardinality	Reading	Example
One-to-one	Each A matches at most one B	A student and that student’s identity card
One-to-many	One A matches many B	One class, many students
Many-to-many	Many A match many B	Students and the books they borrow

Referential integrity means a foreign key must point at a real primary key, or be empty when the design allows that. You cannot record a loan for a student who is not in the student table. **Cascade update** copies a changed primary key into the foreign keys that use it. **Cascade delete** removes the dependent rows when the parent row is deleted. Use

cascade delete only when the dependent rows have no meaning alone. Deleting a student and silently deleting every fee receipt is usually the wrong cascade.

An ER model, including the library pattern

An entity-relationship diagram names the things (rectangles), their attributes (ovals or a list), and the relationships (diamonds or a labelled line). Cardinality is written at the ends of the line.



Member M : N Book becomes two 1 : M links through Issue

Figure 7. The loan is its own entity because the date belongs to the pair, not to the person or the book alone.

Build a schema in this order: list the requirements, name the entities, list attributes, name relationships, mark cardinality, and replace every many-to-many link with a new table that holds both foreign keys. In a library, Member and Book are many-to-many, because one member borrows many books and one book is borrowed by many members over time. Issue (or Loan) stores MemberID, BookID, and the dates. Each foreign key is many-to-one back to its parent.

Objects you build in Access

Table

Stores the rows. Design the keys before you type a hundred records.

Form

A screen for one job, such as registering a student. The form’s data source is a table or a query. The wizard is a fair start; then align the controls.

Query

A question. A select query chooses columns and rows. Criteria on the same row combine with AND. Criteria on different rows combine with OR.

Report

A formatted printout or PDF of a query: a class list, a loan reminder, a summary.

Totals in a query use Group By with Sum, Avg, or Count. A bar or column chart compares categories. A pie chart shows parts of one whole, and only when there are few slices. A line chart shows a value across time. The chart is an argument about the data, so the title should state the finding, not only the word “Chart”.

ONE SENTENCE A relational database keeps each fact in one place and uses keys so a query can reassemble the facts the question needs.

Checkpoint

1. A course has many students and a student enrols in many courses. What extra table does the schema need?
2. Where do you put two criteria so that both must be true?
3. What does cascade delete do, and when is it dangerous?

Sketch answers: an enrolment table with CourseID and StudentID. On the same criteria row. It deletes child rows when the parent disappears; it is dangerous when those rows are records you still need, such as payments.

CLASS XI · LECTURE 5

Applications and impacts of computing

Sensors collect, analytics explain, and AI decides. Then you decide which source to trust.

1. Describe artificial intelligence, the Internet of Things, and data analytics as one pipeline.
2. Give a concrete use of IoT in cities, industry, health, education, homes, energy, farms, and wearables.
3. Tell a reliable information source from an unreliable one.

Artificial intelligence is computing that carries out tasks we associate with noticing, predicting, or choosing. A phone that unlocks on a face, a video site that suggests the next clip, and a site assistant that answers a short question are everyday cases. The system is not wise. It is using data to map an input to an output that used to require a person.

The **Internet of Things** is the network of physical devices that sense the world, send those readings, and sometimes act. A smartwatch, a soil probe, a meter, and a camera become IoT devices when they share data and take part in a task.

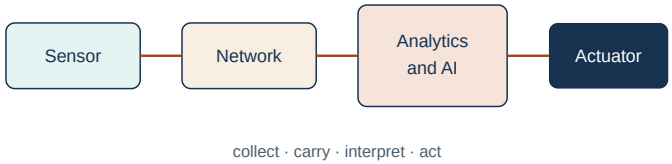


Figure 8. A hot room: the sensor reports, the model decides, the actuator starts the fan.

Four pieces repeat in every example.

- **Sensor.** Temperature, motion, fingerprint, camera, smoke, soil moisture.
- **Connectivity.** Wi-Fi, mobile data, or Bluetooth, so the reading can leave the device.
- **Processing.** Analytics turns readings into a summary. AI uses that summary to choose.
- **Actuator.** A motor, a lock, a valve, or a relay that changes the world.

Data analytics is the work of cleaning, grouping, and reading data so a person or a model can act. IoT without analytics is a pile of readings. Analytics without a question is a chart nobody uses. AI without reliable data repeats the bias in that data.

Setting	What is sensed	What changes
Smart city	Traffic, air, bins, street lights	Signal timing, collection routes
Industry	Vibration, temperature, product checks	Maintenance before a breakdown
Healthcare	Pulse, ward occupancy, medicine stores	Alerts and stock orders
Education	Attendance devices, lab conditions	Records that no longer depend on a paper roll
Home	Motion, smoke, temperature	A door, an alarm, a fan
Smart grid	Meters in homes and on lines	Load shifted away from the peak hour
Farming	Soil moisture, local weather	Irrigation only where the soil is dry
Wearables	Steps, heart rate	A prompt on the wearer’s phone

Reliable and unreliable sources

Lean toward a source when you can name the author or the institution, the date is visible, the method or the data is described, and the claim matches what other serious sources say. Government statistics offices, university pages, standards bodies, and papers that cite their data sit here. A `.gov` or `.edu` address is a clue, not a proof.

Step back when there is no author, no date, and no path to the original numbers. Sensational headlines, undated screenshots, and a shop that is selling the thing it "proves" are weak sources. If two pages disagree, open the table they both claim to quote.

ONE SENTENCE IoT gathers the world's readings, analytics makes them readable, and AI uses the result to recommend or to act.

Checkpoint

1. In an automatic door, which part is the sensor and which part is the actuator?
2. Name two checks you make before you cite a web page in a school report.

Sketch answers: a motion sensor detects a person; the motor that swings the door is the actuator. Name the author or organisation, and check that a date and a source for the figures are visible.

CLASS XI · LECTURE 6

Digital literacy

Ask a sharp question, collect the right data, and present it without taking credit for someone else's work.

1. Distinguish qualitative and quantitative data, and primary and secondary data.
2. Choose among interviews, surveys, prototypes, observation, and simulations.
3. Design a collection plan and present the result in a fitting digital form.
4. Search with operators, and follow consent, privacy, and citation.

Digital literacy is the ability to find, judge, create, and share information with digital tools, safely and with credit. Owning a phone is not the same skill.

Qualitative and quantitative

Qualitative data describes experience in words: why a student prefers printed notes, what a shopkeeper says about closing time. **Quantitative** data is counted or measured: 18 out of 30 students, a download time of 2.4 seconds. Many studies need both. The count says how widespread a problem is. The interview says why it happens.

Primary and secondary

Primary data is collected by you for this question: your survey, your interview, your observation sheet. **Secondary** data was collected by someone else: a census table, a published report, a textbook. Secondary data is faster and sometimes better, because a national sample is larger than a

class. It can also answer a slightly different question from yours. Say which kind you used.

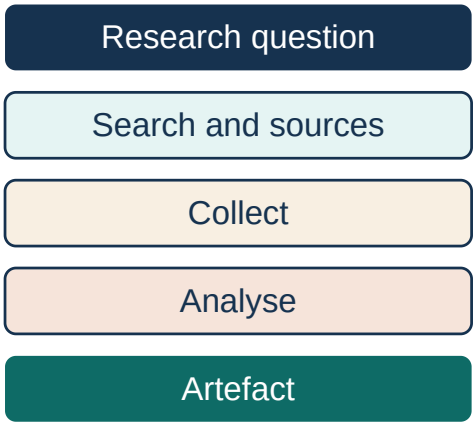


Figure 9. The question chooses the method. The artefact answers the question.

Method	Use it when	Limit
Interview	You need reasons, in the person's own words	Few people, easy to lead the witness
Survey	You need the same questions from many people	Shallow if the questions are vague
Observation	You need what people do, not what they say	Your presence can change the behaviour
Prototype	You need a reaction to an idea you can point at	People comment on the sketch, not on a finished product
Simulation	A real trial is unsafe, slow, or expensive	The result is only as fair as the model

A collection plan

1. Write one question a stranger could understand. "How does phone use during homework relate to time spent on one assignment?" is a question. "Phones are bad" is a slogan.
2. Name the group. "Class XI in this college" is a population you can actually reach.
3. Pick the method from the table, and draft the instrument: five to eight survey items, or an interview guide.
4. Decide the sample. Asking only your friends is a biased sample if the question is about the whole class.
5. State the ethics before you collect anything: what you will ask, whether names are stored, and how someone refuses.

Search operators

A search box is a tool, not a slot for a sentence. Quotes force a phrase. `site:` stays inside one site or kind of site. `filetype:` asks for a format. `OR` allows alternatives. A minus sign excludes a word.

```
"internet usage in Pakistan" filetype:pdf
digital literacy students site:.edu
smartphone homework -advertisement
```

Presenting, and the ethics that sit underneath

A spreadsheet holds the table, computes an average, and draws the chart. Slides are for a spoken explanation with one idea on a slide. An infographic is for a single result a passer-by can read. A report is for the question, the method, the result, and the limit. Pick the form that matches the audience. A chart with no title and no units is decoration.

> **Consent, privacy, citation.** Ask before you record an answer. Collect only what the question needs. Do not publish names if the answers could embarrass a classmate. **Plagiarism** is presenting someone else's words, images, or ideas as your own. Quote, paraphrase, and write the source. A digital artefact that copies three paragraphs from a website has not finished the inquiry.

Checkpoint

1. A student copies a government table of enrolment, then interviews twelve classmates about why they chose the college. Which part is primary, and which is secondary?
2. Write a query that looks for a PDF report on the PTA website about broadband.
3. Why is “Do you agree that our canteen is excellent?” a weak survey item?

Sketch answers: the interviews are primary; the government table is secondary. `broadband Pakistan filetype:pdf site:pta.gov.pk` is one reasonable query. The item leads the respondent and offers no room for a careful answer.

CLASS XII · LECTURE 7

Human–computer interaction

Design the exchange between a person and a system so more people can finish the task.

1. Describe HCI, the human side and the computer side, and where it appears in daily tools.
2. Apply accessibility: contrast, readable type, colour-blind-safe colour, and inclusive navigation.
3. Analyse a user’s need, name common interface failures, and choose a way to improve them.
4. Separate UI from UX, sketch a low-fidelity wireframe, and outline a Figma prototype.

Human–computer interaction is the design of the exchange between people and digital systems. The older names CHI and man–machine interaction point at the same study. A person senses the system, decides, and acts. The system receives that act through an input device, computes, and responds through an output device.

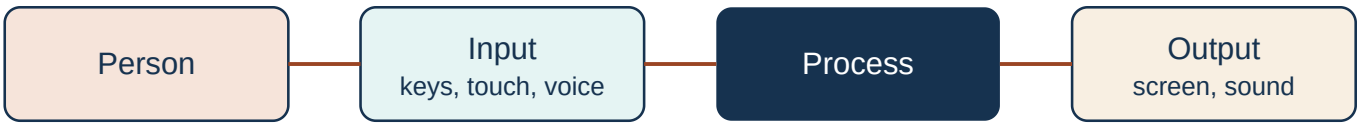


Figure 10. Sight, hearing, and touch are the human channels. Screens, speakers, and vibration are the machine’s reply.

GUI

Windows, icons, menus, pointer. The usual desktop and phone home screen.

Command line

Typed commands. Fast for a person who knows the language, opaque for everyone else.

Touch

The finger is the pointer. Targets must be large enough to hit.

Voice and natural UI

Speech, and interfaces that track a gesture or a headset. Powerful, and easy to design so that only one kind of body can use them.

Traditional interaction expected a trained user at a keyboard. Natural interaction tries to meet the user with speech, touch, and gesture. Both still fail when the task itself was never understood.

Accessibility is part of the design, not a patch

Contrast

Text must stand off the background. Pale grey on white looks delicate and fails in sunlight and for low vision. Dark text on a light surface, or the reverse, is the default.

Readability

A familiar typeface, a size a person can read at arm’s length, generous line height, and short lines. All capitals slow reading.

Colour is not the only signal

Red against green disappears for many colour-blind users. Pair colour with a word, an icon, or a pattern: “Paid” beside a tick, not a green dot alone.

Inclusive navigation

A keyboard path, a visible focus, labels a screen reader can speak, and captions on video. Do not hide the only route inside a gesture.

Needs, failures, and repairs

Need analysis asks who the user is, what they are trying to finish, where they are (a bright courtyard, a noisy lab), and what happens if they make a mistake. Write those answers before you decorate a screen.

Common failures are clutter, low contrast, labels that differ from the user’s words, no confirmation after a tap, and a path that only a mouse can follow. The repairs in this unit are practical: talk to users, sketch, test the sketch, change it, and test again. Continuous testing means a small check every time the interface changes, not a single demonstration on the last day.

UI and UX

The **user interface** is what is on the screen and under the finger: controls, layout, words, colour. The **user experience** is whether the person finishes the task feeling clear about what happened. A beautiful button that submits the form twice is a UI success and a UX failure. Wireframes settle the UX structure before colour arrives.

A **low-fidelity wireframe** is boxes, lines, and real labels. It is supposed to look unfinished, so critics talk about the flow.

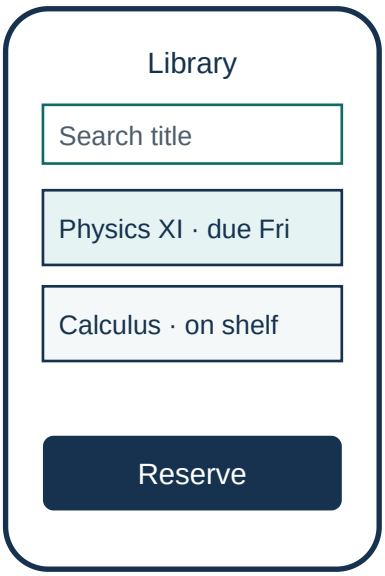


Figure 11. A wireframe commits to words and order, not to a brand colour.

LAB · FIGMA, LOW FIDELITY FIRST

1. Create a free account and a new design file. Press F, or choose the frame tool, and place a phone frame.
2. Draw the blocks of Figure 11 with rectangles and text. Name the layers in the left panel.
3. Duplicate the frame for the next step of the task, such as “Reserved”.
4. Switch to Prototype, drag a link from the button to the next frame, and press Present. Click the path yourself.
5. Ask one classmate to complete the task without your help. Change the label they hesitated on.

Checkpoint

1. A form shows errors only by turning the border red. What accessibility principle does it miss, and what would you add?
2. Why is a wireframe allowed to look rough?

Sketch answers: colour must not be the only signal; add an icon and the words “Date is required”. A rough sketch invites comments on the task. A polished painting invites comments on the shade of blue.

CLASS XII · LECTURE 8

Algorithms, efficiency, and data structures

Prove a small algorithm by tracing it, then store the data in a shape that matches the questions you ask.

1. Check correctness with a trace table and with stepwise reasoning.
2. Judge clarity (modularity, readability) and efficiency (steps, Big O, the problem in front of you).
3. Refine an algorithm, and choose among arrays, linked lists, stacks, queues, trees, and graphs.

Trace tables and stepwise reasoning

An algorithm is correct when every valid input produces the expected output. A **trace table** is a paper execution. Columns hold the line, the variables, the condition, and any input or output. One row is one moment. **Stepwise reasoning** argues in sentences: this line can only increase the counter, the loop ends because the index reaches the length, therefore the counter equals the number of matches.

Count how many scores in $[42, 50, 77, 91]$ lie from 50 to 100 inclusive.

```
count = 0
for score in scores:
    if 50 ⩽ score ⩽ 100:
        count = count + 1
```

score	50 ≤ score ≤ 100	count afterward
42	no	0
50	yes	1
77	yes	2
91	yes	3

The boundary values matter. 50 must be counted because the test is inclusive. A trace that never includes a boundary has not tested the algorithm.

Clarity and Big O

Modularity puts one job in one named piece, so a fault has an address. **Readability** is names, order, and a layout another student can follow next week. Both are clarity. An algorithm can be correct and still be a poor algorithm if nobody can safely change it.

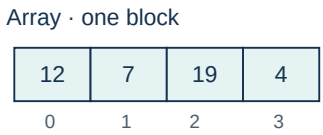
Big O describes how the number of steps grows with the input size n, ignoring constants. It is a comparison tool, not a stopwatch.

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2)$$

Class	Typical cause	Example from this course
$O(1)$	The work does not depend on n	Read array index 0. Push or pop a stack. Enqueue or dequeue when the ends are known.
$O(\log n)$	The problem halves each step	Binary search on a sorted list
$O(n)$	One pass	Linear search, or printing every item
$O(n^2)$	A loop inside a loop over the same data	Bubble sort, selection sort

Also count branches and repetitions in plain language. A nest of special cases can be slow and unreadable even when the Big O looks acceptable. Efficiency includes the real problem: an $O(n^2)$ sort of twenty names is fine; the same sort of a national exam file is not. Refinement renames muddled steps, removes repeated work, extracts a module, or picks a structure whose cheap operation is the one you perform most.

Linear structures



Index 2 is immediate: O(1)

Array. The four values sit in one block. Index 2 is read directly, without walking the earlier cells.



Figure 12. A linked list pays for easy insertion with a walk from the head. There is no index.

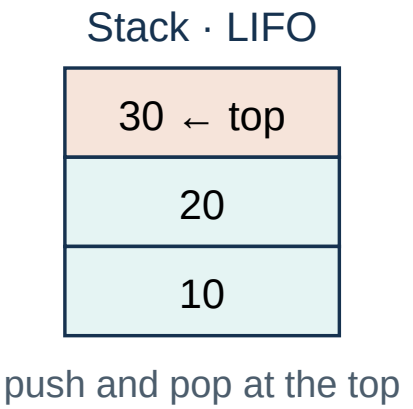


Figure 13. The call stack and undo both behave like a pile of plates.

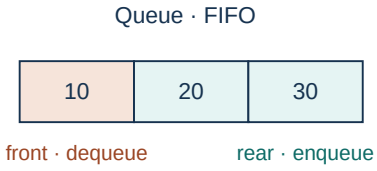


Figure 14. A print queue and a ticket line remove the item that has waited longest.

Trees and graphs

A **tree** organises data by hierarchy. A family tree, folders on a disk, and the staff of a college (principal, then heads,

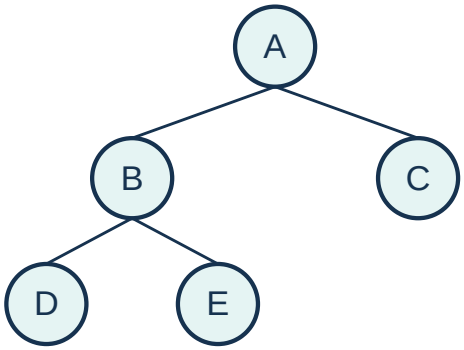
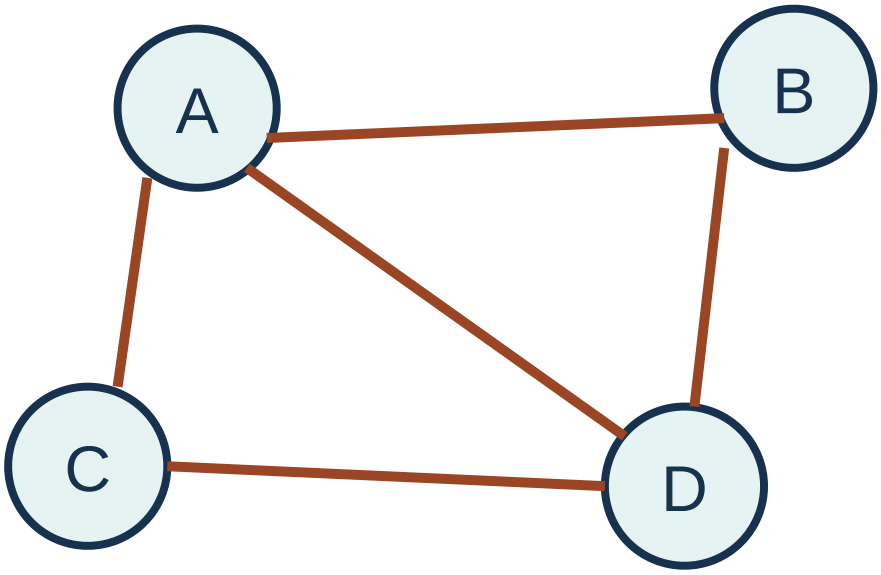


Figure 15. A tree has one root. A is the parent of B and C. D and E are leaves.

then teachers) are trees. Each node except the root has one parent.

A **graph** is a set of vertices joined by edges, with no requirement of a single root. A friendship network and a road map are graphs: Multan may connect to several cities, and the roads can form a loop. Use a tree when “what contains this?” has one answer. Use a graph when connections are many-way.

Arrays suit random access. Linked lists suit frequent inserts and deletes in the middle, if you already hold the place. Stacks suit undo and nested calls. Queues suit fair waiting. Trees suit hierarchies. Graphs suit networks.



A graph. The dots are vertices and the lines are edges. A path can return to where it started, so this is not a tree.

Checkpoint

1. Which structure removes the most recently added item?
2. Why is reading `a[3]` described as $O(1)$, while finding a value in an unsorted array is $O(n)$?
3. A map of bus routes between towns is which structure, and what are the vertices and the edges?

Sketch answers: a stack. The index computes the address directly; a search may have to inspect every cell. A graph: towns are vertices, routes are edges.

CLASS XII · LECTURE 9

Python collections, functions, and files

Choose the collection that matches the data, hide a sub-problem inside a function, and let a file outlive the program.

1. Use lists, tuples, sets, and dictionaries, and say what each one refuses to do.
2. Write functions with parameters and return values, and respect scope.
3. Decompose a problem into functions.
4. Read and write text files.

	Written as	Order	Change later?	Duplicates	Reach an item by
List	`[1, 2, 2]`	Kept	Yes	Yes	Index
Tuple	`(1, 2, 2)`	Kept	No	Yes	Index
Set	`{1, 2}`	Not kept	Yes, add and remove	No	Membership, `in`
Dictionary	`{"a": 1}`	By insertion in modern Python	Yes	Keys are unique	Key

Lists

```
marks = [78, 64, 91, 64]
marks.append(70)
marks[1] = 66
print(marks[0], len(marks))
print(marks[1:3])
```

A slice such as `[1:3]` returns a new list and does not change the original. It starts at index 1 and stops before index 3.

Tuples

```
point = (3, 5)
x, y = point
# point[0] = 9  raises TypeError
```

Use a tuple for a record that must stay fixed: a coordinate, an RGB colour, a row you only want to read. If the tuple holds a list, that inner list can still change. Immutability is about the tuple's own slots.

Sets

A set answers "is this already here?" quickly and supports the algebra you write in mathematics.

```
passed = {11, 14, 18}
late = {14, 22}
print(passed | late)   # union        {11, 14, 18, 22}
print(passed & late)   # intersection {14}
print(passed - late)   # difference   {11, 18}
```

$$A \cup B, \qquad A \cap B, \qquad A \setminus B$$

`discard` removes a member if it is present and stays quiet if it is absent. `remove` raises an error when the member is absent. There is no `set[0]`.

Dictionaries

```
student = {"name": "Hina", "marks": 82}
print(student["name"])
print(student.get("section", "not set"))
student["section"] = "XI-B"
for key in student:
    print(key, student[key])
```

`student["section"]` fails if the key is missing. `get` returns the default instead, which is what you want when you are counting words and a new word should start at zero.

Functions, scope, and decomposition

A function names a sub-problem. `def` creates it. Parameters are the inputs. `return` sends a value back. Without `return`, the caller receives `None`.

```
def grade(marks):
    if marks ⩾ 50:
        return "pass"
    return "fail"

def average(values):
    if len(values) == 0:
        return 0
    return sum(values) / len(values)

scores = [78, 64, 91]
print(grade(scores[0]), average(scores))
```

A name assigned inside a function is **local**. It disappears when the function returns. A name assigned outside is **global**. Read globals sparingly. Pass what the function needs as parameters so the function can be tested on its own. That is decomposition: `average` does not know about grades, and `grade` does not know how the average was computed.

Built-in functions (`len`, `sum`, `print`, `int`) come with the language. The functions you write with `def` are user-defined. Prefer a user-defined function when the same three lines appear twice.

Files

A variable dies when the program ends. A file does not. Use `with` so the file is closed even when something fails halfway.

```
with open("marks.txt", "w", encoding="utf-8") as handle:
    handle.write("Hina,82\n")
    handle.write("Omar,67\n")

with open("marks.txt", "r", encoding="utf-8") as handle:
    for line in handle:
        name, marks = line.strip().split(",")
        print(name, grade(int(marks)))
```

Mode	Effect
`"r"`	Read. The file must already exist.
`"w"`	Write from the start. An existing file is replaced.
`"a"`	Append. New text goes at the end.

> `"w"` is destructive. If the file holds the only copy of the register, open it with `"a"` or write a new file and replace the old one only after the new file is complete.

Checkpoint

1. You need the distinct cities in a list of a thousand addresses. Which collection, and which operator later tests whether "Lahore" is among them?
2. What does the caller receive if a function never executes `return`?
3. Which mode empties a file before writing?

Sketch answers: a set, then `in`. `None`. `"w"`.

CLASS XII · LECTURE 10

Data analysis with Python

Connect to a database, shape a table, draw the right chart, and summarise with a number you can defend.

1. Name the pieces of a Python database connection and run basic SQL through SQLite.
2. Organise rows in a table or DataFrame and build a frequency distribution.
3. Choose a bar chart, histogram, scatter plot, or box plot.
4. Compute mean, median, mode, range, variance, and standard deviation.

Data analysis examines, organises, and interprets data so a decision can cite a result. A school looks at marks. A shop looks at sales. A clinic looks at visits. Python is the glue: it can read a file or a database, compute, and draw.

Connecting to a database

The connection has five parts. The **database** holds the tables. SQLite is a file database that ships with Python and is the right learning tool; MySQL is a server database used when many clients share one store. The **driver** is the software that lets Python speak to that DBMS. The **connection object** opens and closes the session. The **cursor** runs SQL and fetches rows. The **SQL** itself creates, inserts, selects, updates, and deletes.

```
import sqlite3
con = sqlite3.connect("school.db")
cur = con.cursor()
cur.execute("""
    CREATE TABLE IF NOT EXISTS student (
        id INTEGER PRIMARY KEY,
        name TEXT,
        marks INTEGER
    )
""")
cur.execute("INSERT INTO student (name, marks) VALUES (?, ?)", ("Hina", 82))
con.commit()
rows = cur.execute(
    "SELECT name, marks FROM student WHERE marks ⩾ ?", (50,)
).fetchall()
con.close()
print(rows)
```

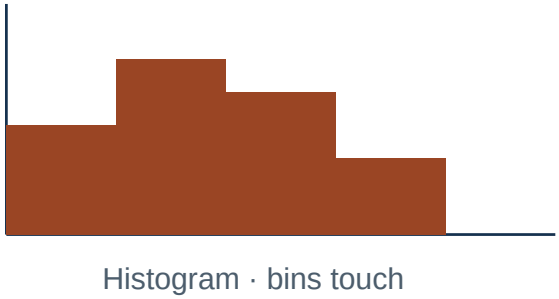
The question marks are parameters. They keep data out of the SQL text. `commit` makes the insert permanent. `close` ends the session. A `with` statement can own the connection in larger programs.

Tables, frequencies, and charts

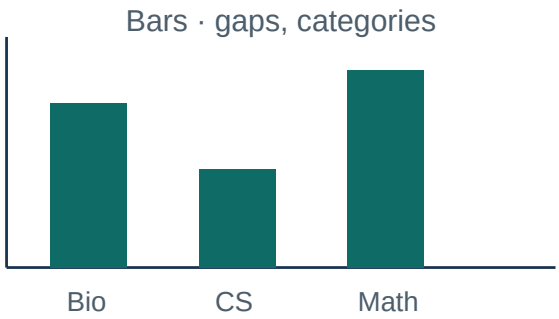
A **DataFrame** (from the pandas library) is a labelled table in memory. You can load it from a query or build it directly. A new column may be an arithmetic combination of old ones, such as `marks / 100`.

```
import pandas as pd
frame = pd.DataFrame({"name": ["Hina", "Omar", "Sana"], "marks": [82, 67, 82]})
print(frame["marks"].mean())
```

A **frequency distribution** groups values into classes and counts each class. Marks 50–59, 60–69, and so on, with a tally column, tell you the shape before any chart does.



Histogram. The bars touch because the axis is one continuous measurement cut into bins.



Bar chart. The gaps keep Bio, CS, and Math as separate categories.

Chart	Use	Python call
Bar	Compare separate categories	`plt.bar(labels, heights)`
Histogram	Distribution of one numeric variable; bars touch because the axis is continuous	`plt.hist(values, bins=...)`
Scatter	Relationship of two numeric variables	`plt.scatter(x, y)`
Box plot	Five-number summary and a glance at spread	`plt.boxplot(values)`

An upward scatter is a positive association. A downward scatter is negative. A round cloud is no clear linear association. Association is not a cause. A box plot shows five numbers: minimum, first quartile Q1, median, third quartile Q3, and maximum. Q1 is a classroom median of the lower half. Q3 is a classroom median of the upper half. The box runs from Q1 to Q3. The line inside the box is the median. The whiskers reach the minimum and the maximum when those points are not treated as outliers.

Central tendency and spread

For values $x_1, \ldots, x_n$,

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \text{Range} = x_{\max} - x_{\min}, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad \sigma = \sqrt{\sigma^2}.$$

This course uses the average of the squared deviations, dividing by n. Some software, including pandas, divides by $n - 1$ unless you say otherwise. Name the choice when you report a variance from a library.

> **Worked numbers.** Values 2, 4, 6. The mean is $\bar{x} = 12/3 = 4$. Deviations are $-2, 0, 2$. Squared deviations are $4, 0, 4$. Variance is $8/3 \approx 2.67$. Standard deviation is $\sqrt{8/3} \approx 1.63$. The range is $6 - 2 = 4$. The median is 4. There is no mode, because no value repeats.

Measure	Prefer it when
Mean	Every value should count, and there is no extreme outlier
Median	A few very large or very small values would drag the mean (salaries, house prices)
Mode	You need the most common category (favourite fruit, most frequent shoe size)

A deviation $x_i - \bar{x}$ is positive above the mean, negative below it, and zero on the mean. Variance squares those deviations so the negatives do not cancel. A small variance means the values sit near the mean. Min and max are the ends of the same story the range compresses into one number.

Checkpoint

1. Which object runs the SQL, the connection or the cursor?
2. Marks of one class, grouped in bands of ten: histogram or bar chart?
3. The salaries of a company include one owner who earns fifty times a typical wage. Which average do you report beside the mean, and why?

Sketch answers: the cursor. A histogram, because the axis is a continuous score cut into bins. The median, because the owner’s wage pulls the mean upward.

CLASS XII · LECTURE 11

Neural networks, security, and equity

How a layered model learns a pattern, how collaboration stays safe, and who gets to take part.

1. Explain machine learning, neural networks, and deep learning, and name their parts.
2. Give applications, including assistive ones.
3. Identify malware, phishing, unauthorised access, data theft, denial of service, and human error, and match a mitigation to each.
4. Explain equity of access in digital collaboration.

A **machine-learning model** is adjusted on examples until its outputs are useful on new examples. A **neural network** is one kind of model. It is inspired by the idea of connected cells, and it is implemented as arithmetic. **Deep learning** trains networks with many hidden layers. Early layers respond to simple pieces (an edge, a short sound). Later layers respond to combinations (an eye, a word). Depth is what makes the hierarchy possible. It is also why these models need a great deal of data and computation.

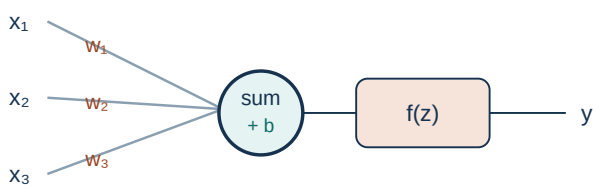


Figure 16. The neuron is a weighted sum, a bias, and a function that shapes the result.

$$z = \sum_{i=1}^{n} w_i x_i + b, \qquad y = f(z)$$

The weights w_i say how much each input matters. The bias b shifts the sum. The activation f decides what leaves the neuron. A step function outputs 1 when $z \geq 0$ and 0 otherwise. A common smooth activation is the sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

which stays between 0 and 1. Training changes the weights and the bias so the outputs move toward the examples. You do not hand-write those numbers for a face recogniser. You do need to know that they are the learned part.

A numeric step. Inputs $1, 0, 1$, weights $0.5, 0.8, -0.2$, bias -0.1. Then $z = 0.5 + 0 + (-0.2) + (-0.1) = 0.2$. A step activation outputs 1. Change the bias to -0.4 and $z = -0.1$, so the step outputs 0. The bias moved the decision.

Applications follow the same shape. Face unlock classifies a camera image. Speech tools turn sound into text. Translation models map one sentence to another. In a clinic, a model can flag a scan for a human to review. Assistive tools read a screen aloud, describe a scene, or convert speech to captions. The responsible pattern is that a person remains accountable for a high-stakes decision.

Secure collaboration

Shared documents, repositories, and class drives are useful because more than one person can touch the work. That is also the risk. Secure collaboration means the right people, the right files, and a way to recover.

Authenticate

A password plus a one-time code is stronger than a password alone. Do not share the password that stands for you.

Encrypt

Encryption turns readable data into ciphertext. Decryption restores it for someone who holds the key. A stolen disk of ciphertext is not yet a stolen essay.

Limit and back up

Give each account only the access the task needs. Keep a second copy, and practise restoring it. A backup you have never restored is a hope.

Firewall and antivirus

A firewall filters network traffic by rules. Antivirus scans files and removable drives for known harmful software. Update both.

Threat	What it is	A fitting mitigation
Malware	Software meant to damage, spy, or lock files: viruses, worms, spyware, ransomware, Trojan horses	Avoid cracked installers, scan downloads, keep antivirus current, hold backups so ransomware is not the only copy
Phishing	A fake message that urges you to hand over a password or an OTP	Check the sender, do not follow urgent links, open the service by typing its address yourself
Unauthorised access	Someone enters an account or a file without permission	Strong unique passwords, lock the screen, do not leave a session open
Data theft and identity theft	Copying personal data, or using it to pretend to be that person	Encryption, least-privilege access, and a refusal to post identity numbers in a shared chat
Denial of service	A flood of fake requests so real users cannot get in	Network filtering, rate limits, and a host that can absorb or block the flood
Human error	The wrong recipient, a deleted folder, a weak password	Confirm the recipient, keep backups, use a checklist before a bulk share

A strong password is long and is not a birthday or the word “student” with a digit. A public lab computer is not a place to store the only copy of a file, and it is not a place to leave an account logged in.

Equity

Equal access means a collaboration tool can be used by the students the task is for, not only by the students with the newest phone. Equity shows up as a file that still opens on a slow connection, a captioned video, a document that a screen reader can follow, language a class can read, and a deadline that does not assume everyone has broadband at home. A tool that only works for one expensive device quietly removes people from the group. Design the task so the work, not the hardware, is what differs.

Checkpoint

1. What extra thing does deep learning have, compared with a network that has a single hidden layer?
2. A message says your class account will close in an hour unless you open a link and type the OTP. Name the threat.
3. Name one design choice that makes a shared assignment fairer for a student on a metered connection.

Sketch answers: many hidden layers that build features in stages. Phishing. A small file, a text alternative to a long video, or a deadline that allows the download to happen at school.

CLASS XII · LECTURE 12

Entrepreneurship in the digital age

Start from a real problem, show a prototype, and learn from the smallest group that might pay attention.

1. Define entrepreneur and entrepreneurship, including digital and local examples.
2. Turn a problem into an idea, and compare low-, mid-, and high-fidelity prototypes.
3. Plan an MVP, name the riskiest assumption, and choose a beachhead market.
4. Collect feedback and say what you will change.

An **entrepreneur** is a person who takes a problem, offers a solution, and accepts the risk that people may not want it. **Entrepreneurship** is that activity. In a digital setting the solution may be an app, a site, a WhatsApp catalogue, or a simple form. The risk and the need to learn from users are the same as they are for a stall.



Figure 17. Each arrow is a decision to keep going, change the idea, or stop.

Local problems are enough. A canteen that never posts today's menu. A tailor whose regular customers cannot see a new cloth until they travel across town. A tuition centre whose parents learn that the van is late only by calling. A student who cannot tell which past paper matches next week's topic. The digital move is to make the missing information or the missing service reachable. It is not to begin with "we need an app".

Prototypes

A **prototype** is an early model you can show. It answers "what would this look like or how would it work?" It does not have to sell, store data, or survive a crowd. Fidelity means closeness to the final product.

	Looks like	Built with	Good for
Low fidelity	Rough	Paper, pencil, a phone photo of a sketch	The first conversation. Cheap to throw away.
Mid fidelity	Clearer layout, still obviously a model	Slides, Figma frames, a clickable series of screens	Testing a path: can someone place an order?
High fidelity	Close to the finished look	Visual design tools, a careful front end	A late check of wording and appearance. Expensive to redo.

Start low. A common failure is to build a high-fidelity app before anyone has confirmed the problem. The cycle is plan, make, show, and improve. Useful questions are specific: "What do you think this button does?" and "What is missing for you to trust the van's location?" "Do you like it?" collects politeness. Negative feedback is the prototype working. Ignoring it is the failure.

The MVP and the riskiest assumption

A **minimum viable product** is the smallest version someone can actually use, so you can learn whether the idea works in the world. A prototype can be paper. An MVP has to function for one narrow job. A menu board in a class WhatsApp group can be an MVP. A full delivery platform is not.

	Prototype	MVP
Purpose	Learn about the design and the problem	Learn whether people use a real, minimal offer
Must it work?	Only well enough to react to	Yes, for one job
Audience	A few users in a test	The first real group, the beachhead

An **assumption** is a belief you have not tested. The **riskiest assumption** is the belief that would make the rest of the plan pointless if it turned out to be wrong. “Parents of this college’s junior classes will open a location message during the commute” is riskier than “we can draw a pleasant icon.” Test the risky belief first. Write the MVP as a short plan: the one job, the features required for that job, a rough cost (time, data, any fee), how you will tell the first users, and how you will collect feedback.

A **beachhead market** is the first small, specific group. Not “students in Pakistan.” “Parents of Class XI in this college who already use the class WhatsApp group.” You can reach them, you can talk to them, and a result there teaches you whether a second group is worth the attempt. After the trial, sort feedback into changes you will make, comments you will decline, and questions you still need to ask. Then change the MVP and, if the beachhead responded, consider the next group. If it did not, change the problem or stop. Stopping is a result.

STUDIO TASK

In one sitting: write a problem you have seen this month, the user, the riskiest assumption, a six-box paper prototype, five feedback questions, the single MVP feature, and the beachhead in one sentence. Show the paper to two people who are in that beachhead. Record what they did, not only what they said.

Checkpoint

1. Why does a college project begin at low fidelity?
2. Give one difference between a prototype and an MVP.
3. Which is a beachhead: “all shops” or “uniform shops on this college’s street”?

Sketch answers: a rough model is cheap to change after the first honest criticism. A prototype explores; an MVP performs one real job for users. The uniform shops on this street, because you can name them and visit them.

APPENDIX

Formula and decision sheet

The relations used in the twelve lectures, in one place.

Truth table

$\text{rows} = 2^n$

Gates

$A \cdot B,\ A + B,\ \overline{A},\ \overline{A \cdot B},\ \overline{A + B},\ A \oplus B$

Search

Linear $O(n)$ on any list. Binary $O(\log n)$ on a sorted list.

Mean and spread

$\bar{x} = \frac{1}{n}\sum x_i,\ \sigma^2 = \frac{1}{n}\sum (x_i - \bar{x})^2$

Neuron

$z = \sum w_i x_i + b,\ y = f(z)$

Data units

Data, segment, packet, frame, bits — top to bottom of OSI.

If the situation is…	Reach for…
Requirements are stable and written	Waterfall
Requirements will move, and users can try a slice	Agile, in sprints
Many-to-many in an ER diagram	A new table of the two foreign keys
Undo, or a function call waiting to return	Stack
Fair waiting	Queue
Hierarchy	Tree
Roads, friendships, arbitrary links	Graph
A fixed pair of values	Tuple
Unique members and unions	Set
Lookup by a name	Dictionary
One numeric distribution	Histogram or box plot
Two numeric variables together	Scatter plot
A belief that could sink the idea	Test it with the MVP, on the beachhead

Before a test, redraw a gate, the OSI stack, one sort, an ER link, the IoT pipeline, a wireframe, a stack, a queue, a neuron, and the path from problem to beachhead.